

Artificial Intelligence (AI-2002)

Lecture 7: Search Algorithms

Mr. Saif ur Rehman (Lecturer – AI & DS)

saifurrehman@nu.edu.pk



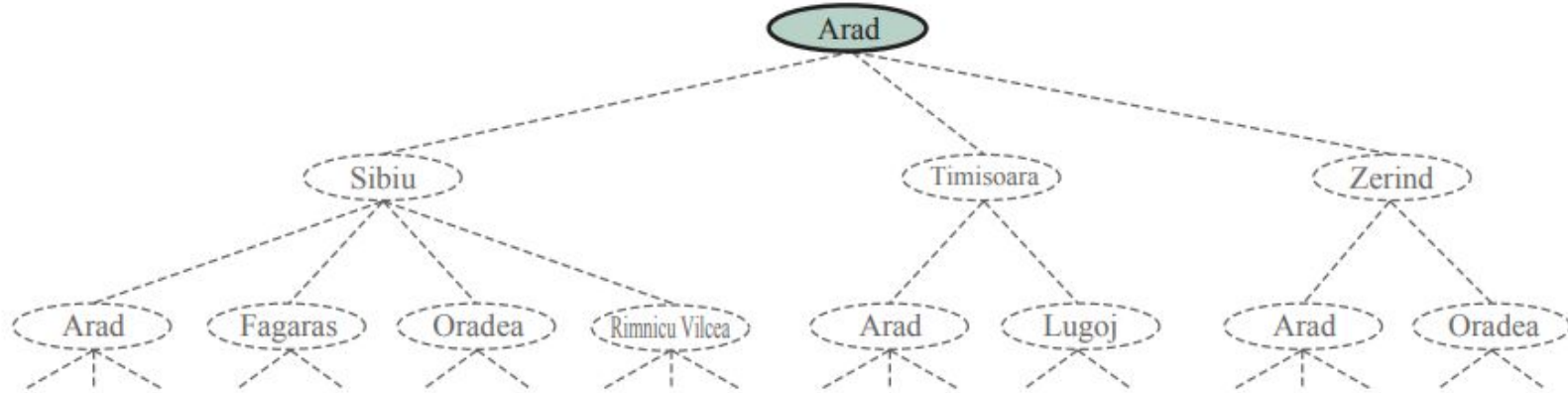
Search Algorithms

- A search algorithm takes a search problem as input and returns a solution or indicates failure.
- The algorithm uses a search tree over the state-space graph.
- Each node in the tree represents a state in the state space.
- Edges in the tree represent actions.
- The root of the tree corresponds to the initial state of the problem.
- The algorithm explores paths to reach a goal state.

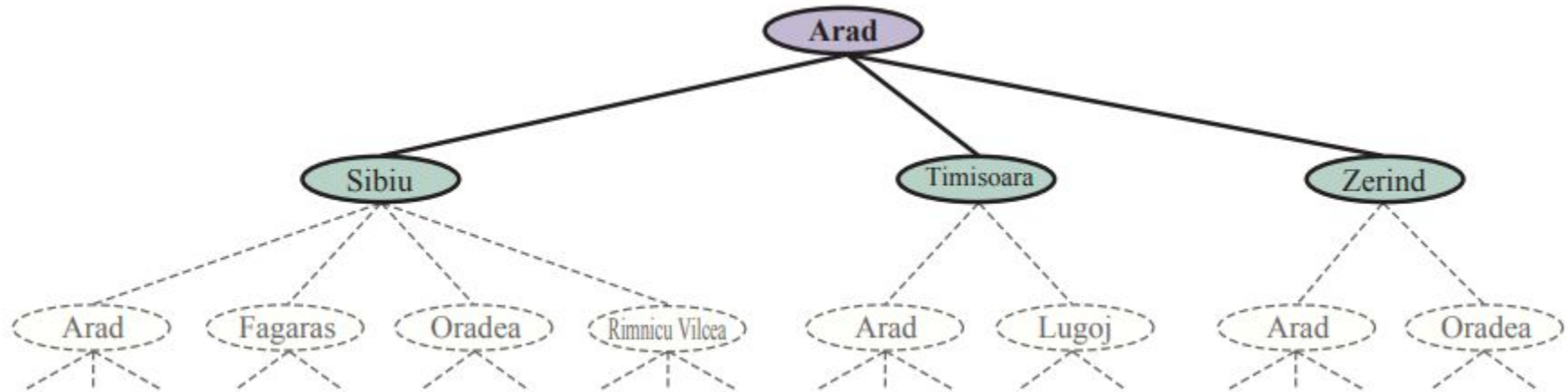
Search Algorithms

- State space: Describes the set of states and actions that allow transitions between them.
- Search tree: Represents paths between states, moving towards the goal.
- The search tree may have multiple paths and nodes for a given state.
- Each node in the search tree has a unique path back to the root (as in all trees).

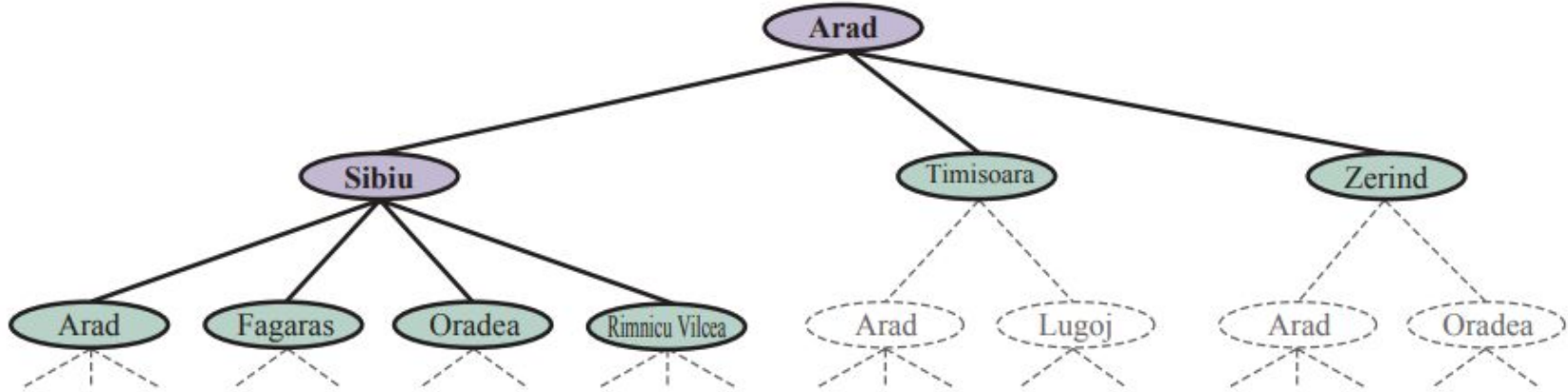
Search Algorithms



Search Algorithms



Search Algorithms



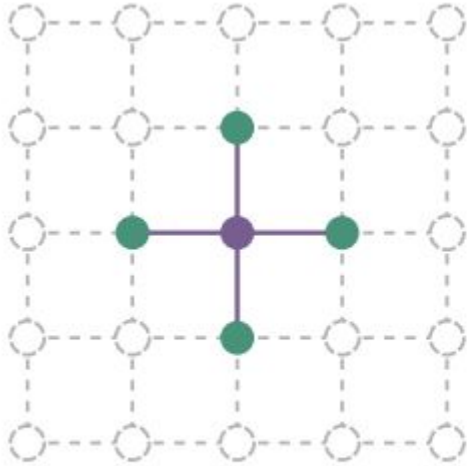
Search Algorithms

Three partial search trees for finding a route from Arad to Bucharest. Nodes that have been expanded are lavender with bold letters; nodes on the frontier that have been generated but not yet expanded are in green; the set of states corresponding to these two types of nodes are said to have been reached. Nodes that could be generated next are shown in faint dashed lines. Notice in the bottom tree there is a cycle from Arad to Sibiu to Arad; that can't be an optimal path, so search should not continue from there.

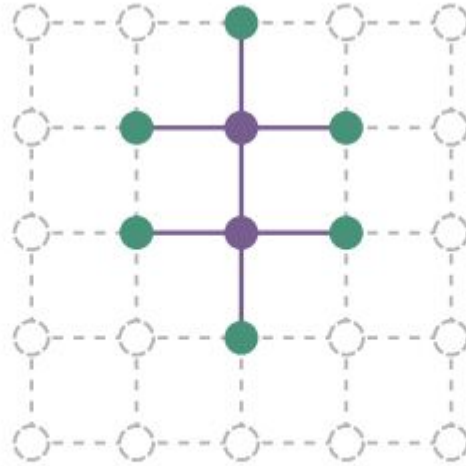
Search Algorithms

- The essence of search is choosing which child node to explore next.
- One option is pursued now, while the others are put aside for later consideration.

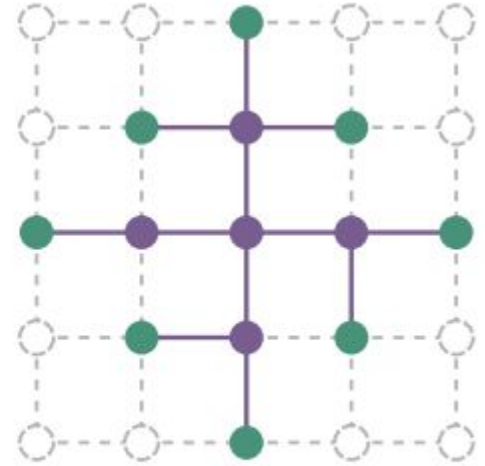
Search Algorithms



(a)



(b)



(c)

Note that the frontier separates two regions of the state-space graph: an interior region where every state has been expanded, and an exterior region of states that have not yet been reached.

Best-first search

- Best-first search: Chooses a node with the minimum value of an evaluation function, $f(n)$.
- On each iteration, select a node from the frontier with the lowest $f(n)$.
- If the node's state is a goal, return it; otherwise, expand to generate child nodes.
- Child nodes are added to the frontier if not previously visited, or re-added if a lower-cost path is found.
- The algorithm returns either a path to the goal or an indication of failure, with variations based on the $f(n)$ function.

Search data structures

Search algorithms require a data structure to keep track of the search tree. A node in the tree is represented by a data structure with four components:

- `node.STATE`: the state to which the node corresponds;
- `node.PARENT`: the node in the tree that generated this node;
- `node.ACTION`: the action that was applied to the parent's state to generate this node;
- `node.PATH-COST`: the total cost of the path from the initial state to this node. In mathematical formulas, we use $g(\text{node})$ as a synonym for `PATH-COST`.

Search data structures

We need a data structure to store the frontier. The appropriate choice is a queue of some kind. Three kinds of queues are used in search algorithms:

- A **priority queue** first pops the node with the minimum cost according to some evaluation function, f . It is used in best-first search.
- A **FIFO queue** or first-in-first-out queue first pops the node that was added to the queue first; we shall see it is used in breadth-first search.
- A **LIFO queue** or last-in-first-out queue (also known as a stack) pops first the most recently added node; we shall see it is used in depth-first search.

Search data structures

- The reached states can be stored as a lookup table (e.g. a hash table) where each key is a state and each value is the node for that state.

Search data structures

- **Repeated state:** A state that appears more than once in the search tree, often caused by cycles (loopy paths).
- **Cycle:** A special case of a redundant path, where a state can be revisited infinitely in the search tree.
- **Redundant path:** A longer or less optimal path to the same state, such as the path Arad–Zerind–Oradea–Sibiu compared to Arad–Sibiu.
- Redundant paths do not need to be considered in the search for optimal solutions.

Measuring problem-solving performance

Before we get into the design of various search algorithms, performance will be evaluated using four criteria:

- **Completeness:** Is the algorithm guaranteed to find a solution when there is one, and to Completeness correctly report failure when there is not?
- **Cost optimality:** Does it find a solution with the lowest path cost of all solutions?
- **Time complexity:** How long does it take to find a solution? This can be measured in seconds, or more abstractly by the number of states and actions considered.
- **Space complexity:** How much memory is needed to perform the search?

Measuring problem-solving performance

- Completeness in finite state spaces achieved by systematically exploring all reachable states, cutting off cycles.
- Completeness in infinite state spaces requires careful exploration to avoid infinite paths that never revisit previous states

Measuring problem-solving performance

Time and space complexity can be measured using:

- **d**: The depth or number of actions in an optimal solution.
- **m**: The maximum number of actions in any path.
- **b**: The branching factor, or the number of successors of a node to be considered.

Uninformed Search Strategies

- **Uninformed search:** No knowledge about how close a state is to the goal. The algorithm has no clue which action will bring the agent closer to the goal (e.g., choosing between Zerind or Sibiu without geographical knowledge).
- **Informed search:** The agent has additional knowledge (e.g., knowing that Sibiu is closer to Bucharest), which helps make better decisions towards reaching the goal.

Breadth-first search

- **Breadth-first search (BFS):** Used when all actions have the same cost.
- The root node is expanded first, followed by the successors of the root, and so on.
- It is a systematic search strategy, ensuring that every level of the state space is explored before moving to the next level.
- **Complete:** BFS guarantees finding a solution, even in infinite state spaces, as it explores all possible states level by level.

Breadth-first search

- Breadth-first search uses a first-in-first-out (FIFO) queue, which will be faster than a priority queue and will ensure the correct order of node exploration.
- BFS is cost-optimal for problems where all actions have the same cost.
- BFS is complete even when actions have different costs.
- In a uniform tree with branching factor b , BFS generates:

$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$$

Breadth-first search

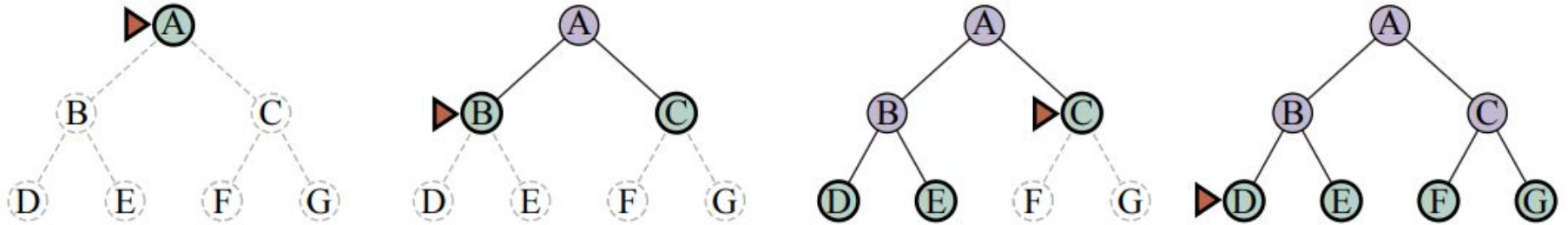
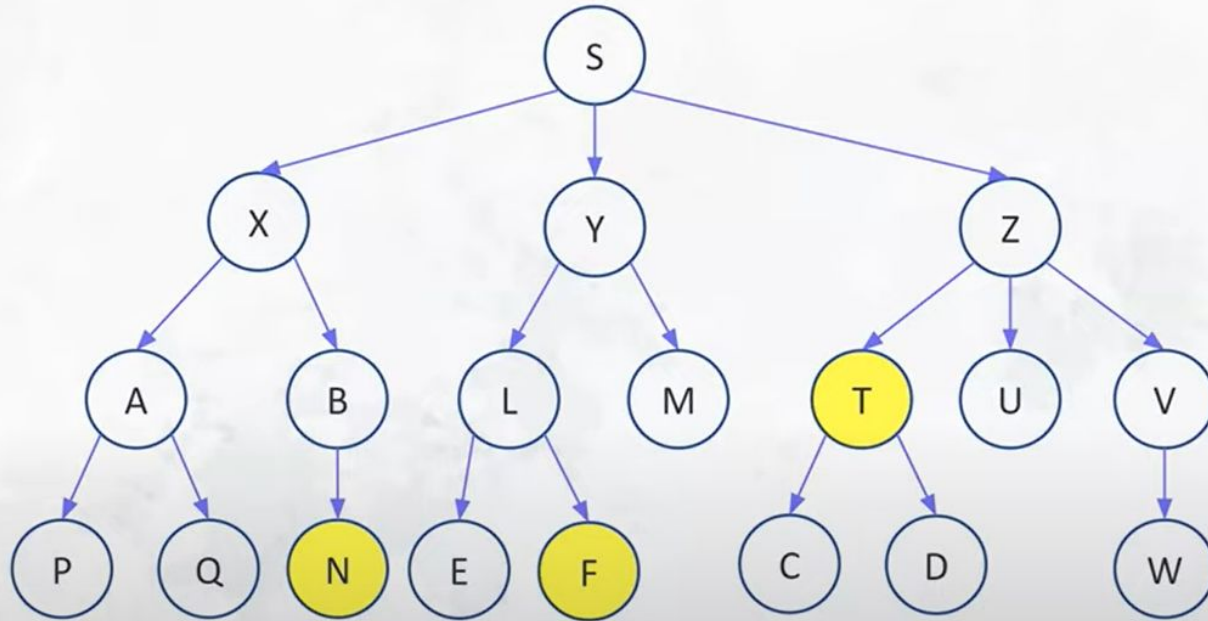


Figure 3.8 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by the triangular marker.

Breadth-First Search (BFS)



node	frontier
	S
S	XYZ
X	YZAB
Y	ZABLM
Z	ABLMTUV
A	BLMTUVPQ
B	LMTUVPQN
L	MTUVPQNEF
M	TUVPQNEF
T	UVPQNEFCD

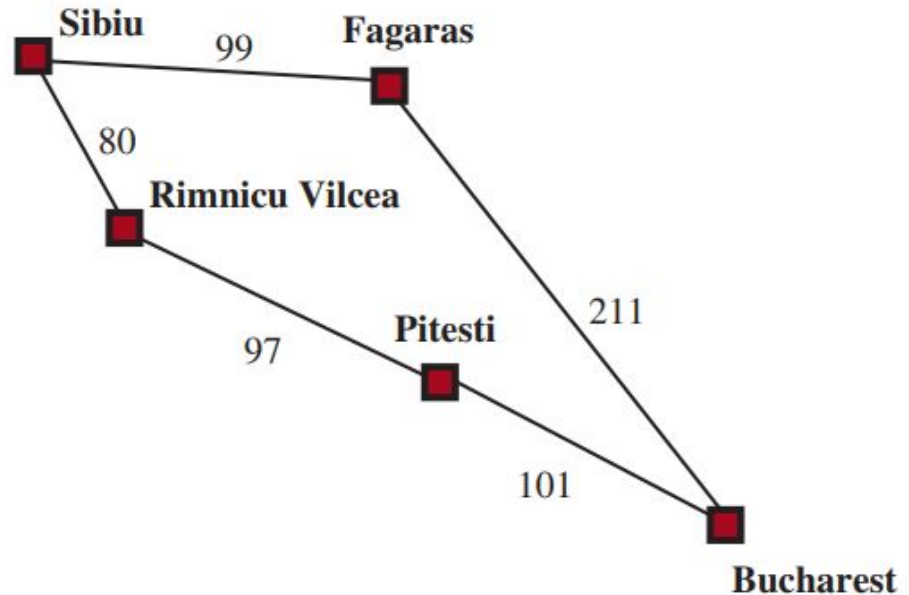
Big O notation is used to
measure how running time or
space requirements for your
program grow as input size
grows

Breadth-first search

- BFS requires $O(b^d)$ memory and time, where b is the branching factor and d is the depth.
- For a branching factor $b=10$, processing speed of 1 million nodes/second, and memory of 1 Kbyte per node: A search to depth $d=10$ takes less than 3 hours but requires 10 terabytes of memory. At depth $d=14$, even with infinite memory, the search would take 3.5 years.
- Due to exponential growth in time and memory requirements, uninformed search is impractical for most real-world problems beyond small instances.

Dijkstra's algorithm or uniform-cost search

- **Uniform-cost search (UCS):**
A variant of best-first search where the evaluation function is the path cost from the root to the current node. Also known as Dijkstra's algorithm.



Dijkstra's algorithm or uniform-cost search

- The successors of Sibiu are Rimnicu Vilcea and Fagaras, with costs 80 and 99, respectively.
- The least cost node, Rimnicu Vilcea, is expanded next, adding Pitesti with cost $80 + 97 = 177$.
- The least-cost node is now Fagaras, so it is expanded, adding Bucharest with cost $99 + 211 = 310$.
- The algorithm continues on, choosing Pitesti for expansion next and adding a second path to Bucharest with cost $80 + 97 + 101 = 278$.

Dijkstra's algorithm or uniform-cost search

- The complexity of uniform-cost search is characterized in terms of C^* , the cost of the optimal solution, and ϵ , a lower bound on the cost of each action, with $\epsilon > 0$. Then the algorithm's worst-case time and space complexity is $O(b^{1+\lceil C^*/\epsilon \rceil})$.
- Which can be much greater than b^d

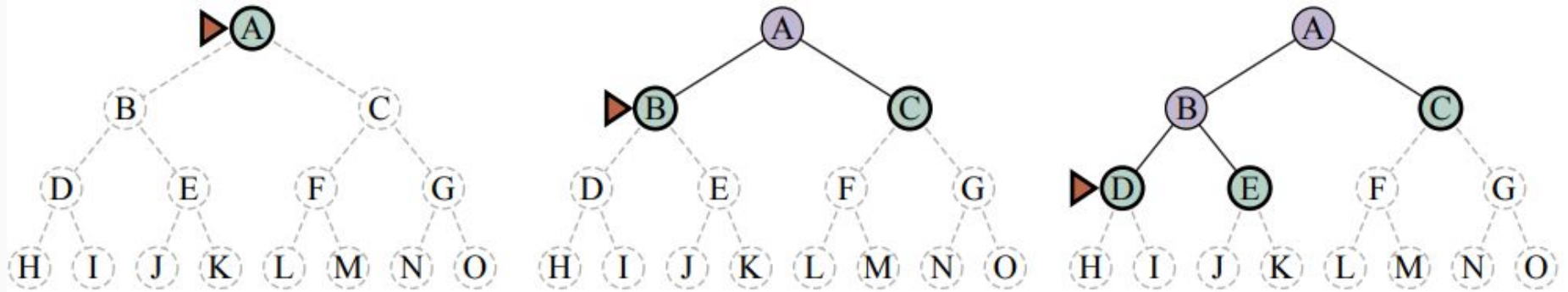
Dijkstra's algorithm or uniform-cost search

- **Completeness:** Guaranteed to find a solution if one exists (all action costs $> \epsilon > 0$).
- **Cost-Optimality:** Always finds the lowest-cost solution first.
- **Avoids Infinite Paths:** Never gets stuck in an infinite branch due to cost-based expansion.

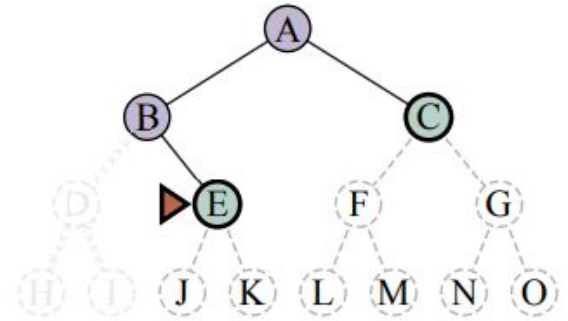
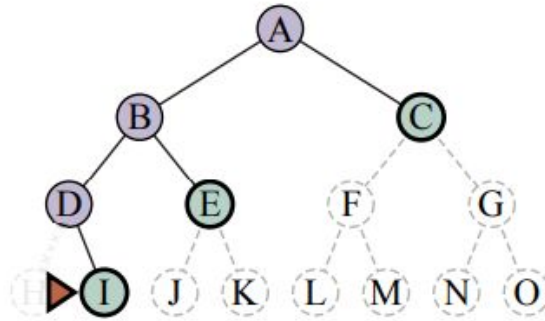
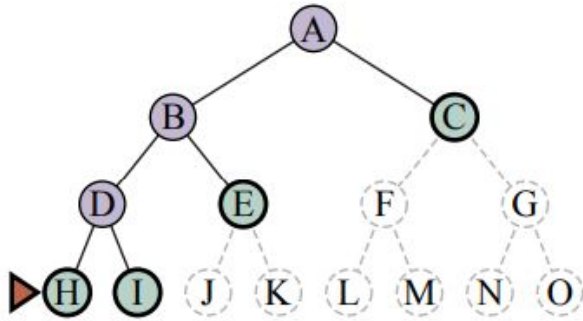
Depth-first search and the problem of memory

- Depth-first search always expands the deepest node in the frontier first.
- Search proceeds immediately to the deepest level of the search tree, where the nodes have no successors.
- The search then “backs up” to the next deepest node that still has unexpanded successors.

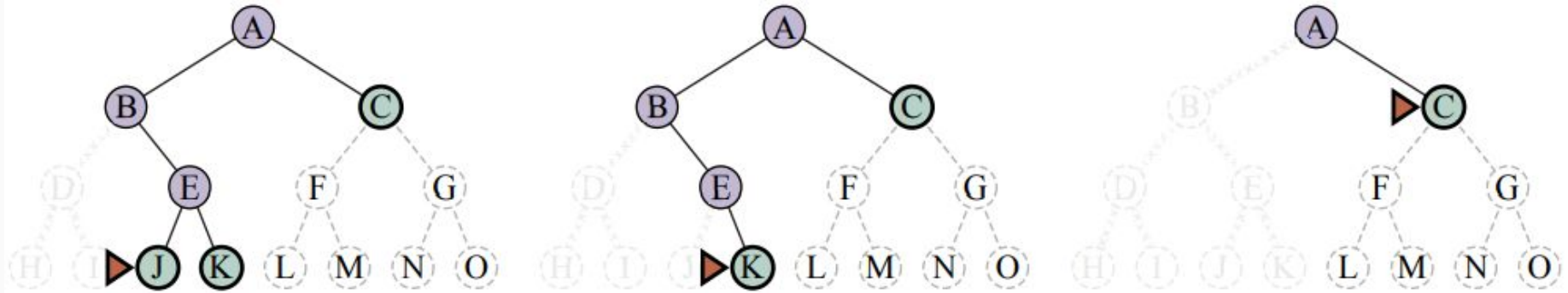
Depth-first search and the problem of memory



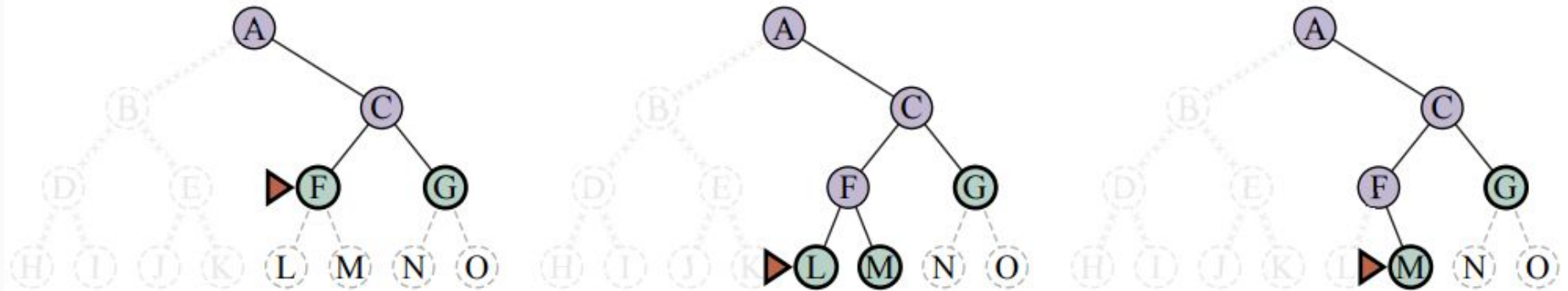
Depth-first search and the problem of memory

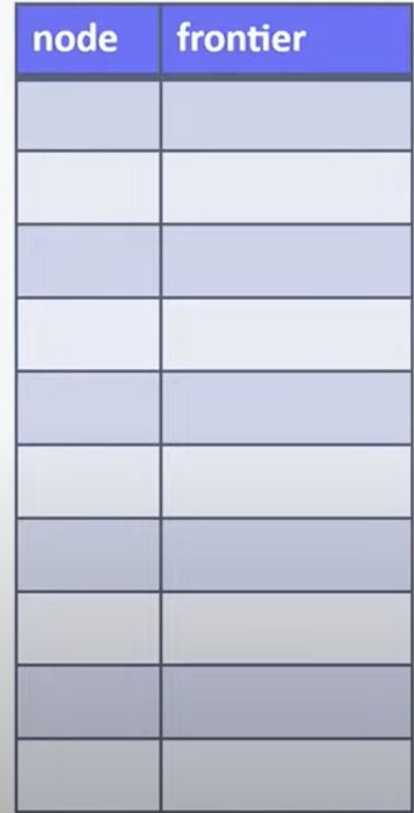


Depth-first search and the problem of memory



Depth-first search and the problem of memory



[illegible]

Depth-first search and the problem of memory

- **Not cost-optimal:** Returns the first solution found, regardless of cost.
- **Efficient and complete for finite state spaces (trees):** Explores the space systematically.
- **For acyclic state spaces:** May expand the same state multiple times but eventually explores all states.
- **For cyclic state spaces:** Can get stuck in infinite loops
- **For infinite state spaces:** Not systematic; can get stuck on infinite paths, making it incomplete.

Depth-first search and the problem of memory

With all this bad news, why would anyone consider using depth-first search rather than breadth-first or best-first?

- depth-first search has much smaller needs for memory as the frontier is very small
- **Time complexity:** Proportional to the number of states in the state-space $O(b^m)$
- **Space complexity:** $O(bm)$, where: b = branching factor, m = maximum depth of the tree.
- **Memory efficiency:** Depth-first search can handle problems requiring exabytes of memory with breadth-first search using only kilobytes.

Depth-limited and iterative deepening search

- Depth-limited search: A variation of depth-first search with a user-supplied depth limit ℓ .
- The time complexity is $O(b^\ell)$.
- space complexity is $O(b\ell)$.
- Unfortunately, if we make a poor choice for the algorithm will fail to reach the solution, making it incomplete again.

Depth-limited and iterative deepening search

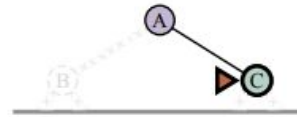
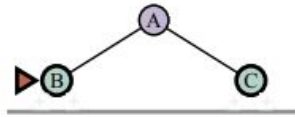
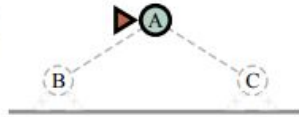
- Iterative deepening search: Solves the problem of choosing a good depth limit by iteratively deepening the search level, starting from 0, then 1, 2, and so on.
- Time complexity: $O(b^d)$ when a solution is found and $O(b^m)$ when there is no solution.
- $O(bd)$ when there is a solution, or $O(bm)$ on finite state spaces with no solution.
- Memory-efficient like depth-first search, as it doesn't store all nodes in memory.
- Optimal and complete like breadth-first search when all actions have the same cost and the state space is finite and acyclic

Depth-limited and iterative deepening search

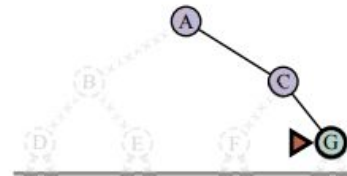
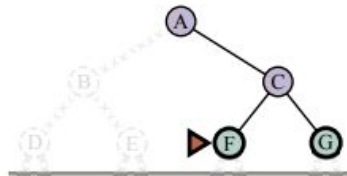
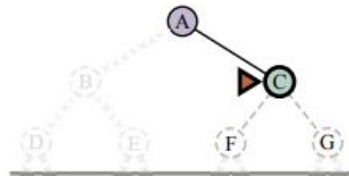
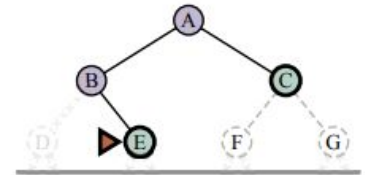
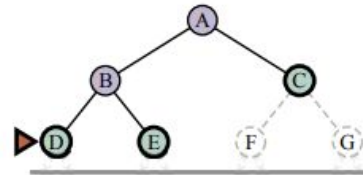
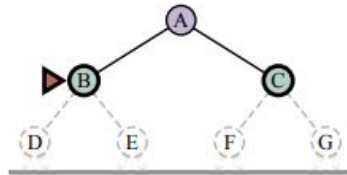
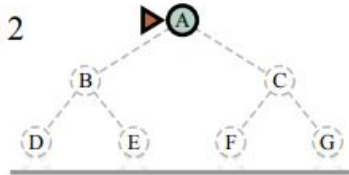
limit: 0



limit: 1

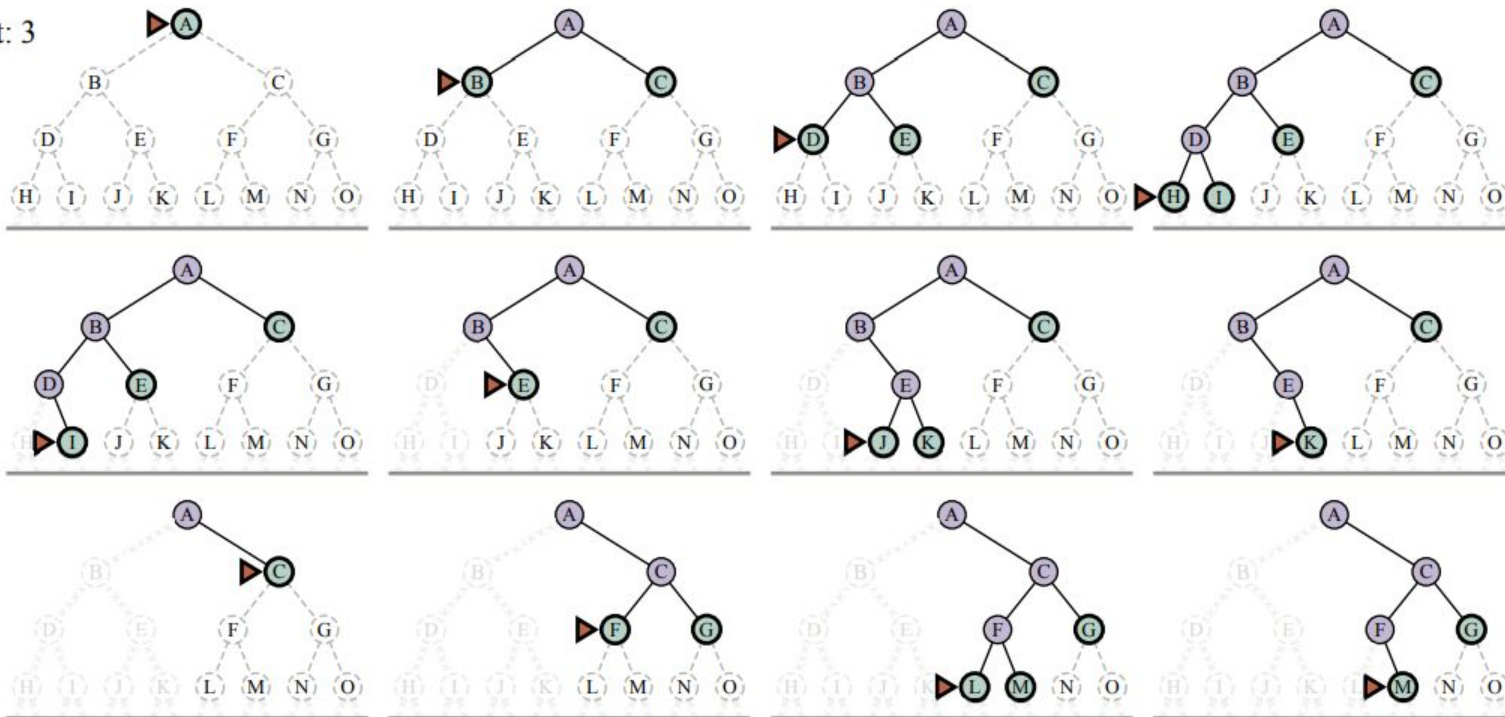


limit: 2



Depth-limited and iterative deepening search

limit: 3



Bidirectional search

- Searches both forward from the initial state and backward from the goal state(s) simultaneously, hoping the two searches meet in the middle.
- The motivation is that $b^{d/2} + b^{d/2}$ is much less than b^d (e.g., 50,000 times less when $b=d=10$).
- There are many different versions of bidirectional search, just as there are many different unidirectional search algorithms.

Comparing uninformed search algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?						
Optimal cost?						
Time						
Space						

Comparing uninformed search algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Figure 3.15 Evaluation of search algorithms. b is the branching factor; m is the maximum depth of the search tree; d is the depth of the shallowest solution, or is m when there is no solution; ℓ is the depth limit. Superscript caveats are as follows: ¹ complete if b is finite, and the state space either has a solution or is finite. ² complete if all action costs are $\geq \epsilon > 0$; ³ cost-optimal if action costs are all identical; ⁴ if both directions are breadth-first or uniform-cost.